Perfect question ⍰ Apache Tomcat setup + deployment steps are slightly different on **Windows 11** and **Ubuntu 22.04**, so I'll give you everything in a **step-by-step comparison table** and also a **visual directory structure diagram** for clarity.

Step-by-Step Installation & Deployment

Step	Windows 11	Ubuntu 22.04
1. Install Java	Download JDK from Oracle/AdoptOpenJDK → Install → Set JAVA_HOME in Environment Variables	sudo apt update && sudo apt install openjdk-11-jdk -y → Check with java –version
2. Download Tomcat	Go to Tomcat official site → Download .zip (Core Binary)	wget https://downloads.apache.org/tomcat/tomcat-9/v9.0.xx/bin/apache-tomcat-9.0.xx.tar.gz
3. Extract Tomcat	Unzip into C:\apache-tomcat-9.0.xx\	tar -xvzf apache-tomcat-9.0.xx.tar.gz → move to /opt/tomcat
4. Set Environment Variables (optional)	Add CATALINA_HOME = C:\apache-tomcat-9.0.xx in System Env Vars	Export inside /etc/environment: CATALINA_HOME=/opt/tomcat
5. Start Tomcat	Run bin\startup.bat (double-click or in cmd)	cd /opt/tomcat/bin && ./startup.sh
6. Verify Tomcat	Open browser → http://localhost:8080	Open browser → http://your-server-ip:8080
7. Deploy App	Copy .war file into C:\apache-tomcat-9.0.xx\webapps\ → Tomcat auto-deploys	Copy .war file into /opt/tomcat/webapps/ → Tomcat auto-deploys
8. Access Application	Example: http://localhost:8080/myapp	Example: http://your-server-ip:8080/myapp
9. Stop Tomcat	Run bin\shutdown.bat	Run ./shutdown.sh inside /opt/tomcat/bin/

Apache Tomcat Directory Structure (Visual Diagram)

Here's the **essential structure** you'll see after extracting Tomcat:

```
apache-tomcat-9.0.xx/
│
├── bin/            # startup.sh, shutdown.sh, *.bat scripts
├── conf/           # server.xml, web.xml (config files)
├── lib/            # JAR libraries required by Tomcat
├── logs/           # Tomcat logs
├── temp/           # Temp files
├── webapps/        # Deploy apps here (WAR or extracted folders)
│   ├── ROOT/       # Default root application
│   ├── manager/    # Web-based Tomcat Manager
│   ├── host-manager/  # Host manager
│   └── myapp.war   # <-- Your deployed application
├── work/           # Compiled JSP files
└── LICENSE, README, etc.
```

With this, you can install Tomcat on **both Windows and Ubuntu**, start it, and deploy any .war application.

Do you want me to also show you **how to create a sample maven-web-app.war and deploy it on Tomcat step-by-step**? That way you'll get a full real-world demo.

Perfect ⚙️ Let's do a **real-world demo**: we'll create a **Maven Web App**, package it into a .war, and deploy it on **Apache Tomcat** (both Windows 11 & Ubuntu 22.04).

Step 1: Install Prerequisites

Tool	Windows 11	Ubuntu 22.04
Java (JDK 11+)	Install from AdoptOpenJDK → Add JAVA_HOME in environment variables	sudo apt install openjdk-11-jdk –y
Maven	Download Maven .zip → Extract → Add MAVEN_HOME	sudo apt install maven –y
Tomcat 9/10	Download .zip → Extract to C:\apache-tomcat-9.0.xx\	wget ... && tar -xvzf ... && mv apache-tomcat-9.0.xx /opt/tomcat

---

Step 2: Create Maven Web Project

Run in terminal / cmd:

```
mvn archetype:generate -DgroupId=com.example \
  -DartifactId=maven-web-app \
  -DarchetypeArtifactId=maven-archetype-webapp \
  -DinteractiveMode=false
```

This creates a project:

```
maven-web-app/
│
├── pom.xml
├── src/
│   └── main/
```

```
│   ├── java/        # Java source code (if any)
│   └── webapp/      # Web content (HTML, JSP, WEB-INF)
│       ├── index.jsp
│       └── WEB-INF/
│           └── web.xml
```

Step 3: Update pom.xml

Here's a **minimal example** of pom.xml:

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
                  http://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>
  <groupId>com.example</groupId>
  <artifactId>maven-web-app</artifactId>
  <version>1.0-SNAPSHOT</version>
  <packaging>war</packaging>

  <build>
    <plugins>
      <!-- WAR packaging plugin -->
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-war-plugin</artifactId>
        <version>3.3.2</version>
      </plugin>
    </plugins>
  </build>
</project>
```

Step 4: Add index.jsp

Inside src/main/webapp/index.jsp:

```
<html>
  <body>
    <h1>Hello, Tomcat + Maven!</h1>
  </body>
</html>
```

Step 5: Build WAR

Run:

```
cd maven-web-app
mvn clean package
```

Output:

target/maven-web-app.war

Step 6: Deploy WAR to Tomcat

Windows 11	**Ubuntu 22.04**
Copy target/maven-web-app.war → C:\apache-tomcat-9.0.xx\webapps	sudo cp target/maven-web-app.war /opt/tomcat/webapps/
Start Tomcat: bin\startup.bat	Start Tomcat: /opt/tomcat/bin/startup.sh
Open: http://localhost:8080/maven-web-app	Open: http://<server-ip>:8080/maven-web-app

---

Deployment Directory Structure

```
apache-tomcat-9.0.xx/
└── webapps/
    ├── ROOT/
    ├── manager/
    ├── host-manager/
    └── maven-web-app.war    <-- deployed WAR
```

After deployment, Tomcat will extract it:

```
apache-tomcat-9.0.xx/webapps/maven-web-app/
│
├── index.jsp
└── WEB-INF/
    └── web.xml
```

At this point, your **Maven Web App is live on Tomcat** □

Do you want me to also show you how to **automate deployment** (so you don't have to manually copy .war) using **Tomcat Maven Plugin** or **Jenkins CI/CD**?

Got it ⍰ Let's go through the **Apache Tomcat Life Cycle** step by step.

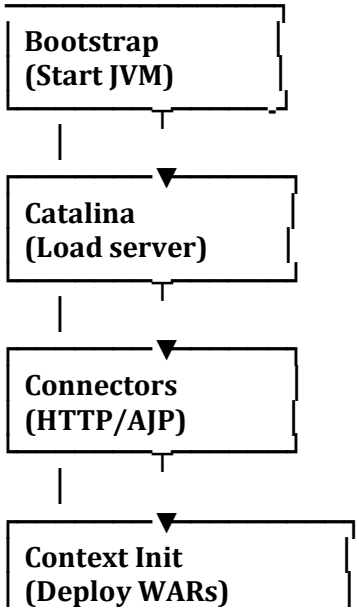
Apache Tomcat is a **Servlet Container**, so its lifecycle revolves around **server startup → request handling → shutdown**.

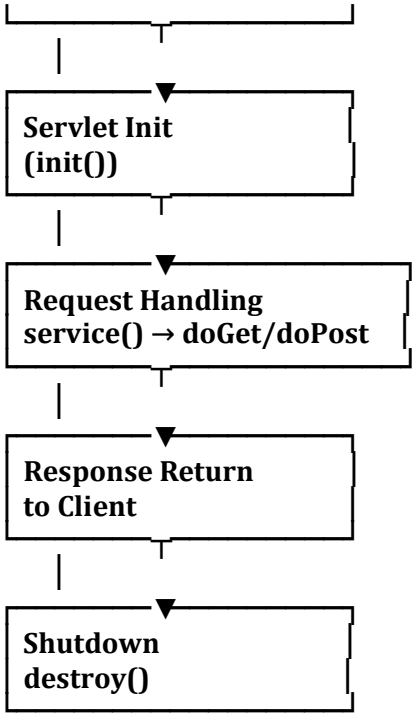
Tomcat Life Cycle Stages

Stage	Description	Key Components
1. Initialization (Bootstrap)	When Tomcat is started, the **Bootstrap class** initializes the JVM and loads system properties.	Bootstrap.java
2. Catalina Startup	The **Catalina** component (Tomcat's core) is started. It parses server.xml and initializes **Server → Service → Connector → Engine → Host → Context** hierarchy.	Catalina.java, server.xml
3. Connector Initialization	Tomcat sets up **Connectors** (HTTP/HTTPS, AJP) to listen for client requests on specific ports (default 8080).	Connector, ProtocolHandler
4. Context Initialization (Deployment)	Each deployed **web application** (.war or exploded folder) gets a Context. Tomcat loads web.xml, initializes **Servlets**, **Filters**, and **Listeners**.	StandardContext, web.xml
5. Servlet Initialization	For each servlet: init() is called once before handling requests.	Servlet.init()
6. Request Processing	For every request: - Connector receives HTTP request - Passes it to **Engine → Host → Context → Wrapper** - Servlet's service() method is executed (dispatching to doGet(), doPost(), etc.)	Servlet.service(), doGet(), doPost()

7. Response Generation	Servlet generates a response (HTML, JSON, etc.) → Connector sends it back to client (browser).	HTTP Connector
8. Servlet Destruction	On undeploy/shutdown, destroy() is called once for each servlet to release resources.	Servlet.destroy()
9. Context Shutdown	The web application context is destroyed, sessions are invalidated, and resources cleaned.	StandardContext.stop()
10. Catalina Shutdown	When Tomcat is stopped (shutdown.sh / shutdown.bat), Catalina stops all services and closes connectors.	Catalina.stop()

Tomcat Lifecycle Flow Diagram





Servlet Lifecycle in Tomcat

Inside a deployed app, **each servlet** follows its own mini lifecycle:

1. **Load & Instantiate** → Servlet class loaded by classloader.
2. **init()** → Called once at startup.
3. **service()** → Called for each request (doGet, doPost).
4. **destroy()** → Called once when app is undeployed or Tomcat shuts down.

This is the **complete Tomcat lifecycle**:

- **Tomcat lifecycle** (server level)
- **Servlet lifecycle** (application level)

Would you like me to also explain **Tomcat architecture components** (Catalina, Coyote, Jasper, etc.) with a diagram, so you can connect lifecycle to architecture?

Great question ⍰ Let's break down the **Apache Tomcat Architecture Components** and then show a **visual diagram** so you can see how everything connects.

Tomcat Architecture Components

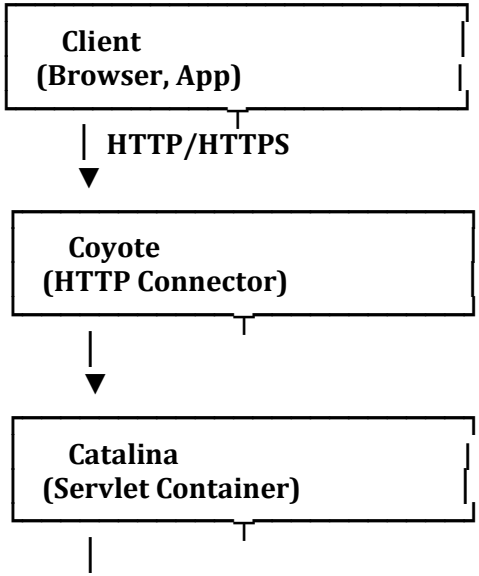
Component	Role
Catalina	The **Servlet Container** – core engine of Tomcat. It manages the lifecycle of Servlets, JSPs, and web applications.
Coyote	The **HTTP Connector** – handles HTTP/HTTPS/AJP requests from clients and forwards them to Catalina.
Jasper	The **JSP Engine** – compiles JSP files into servlets and executes them.
Cluster	Provides **load balancing and session replication** across multiple Tomcat instances.
Naming (JNDI)	Provides access to resources like JDBC DataSources, mail sessions, JMS, etc.
Valves & Realms	Security and authentication components – handle request filtering, authentication, and authorization.
Server.xml (Configuration)	Defines how Catalina, Coyote, Hosts, and Connectors work together.

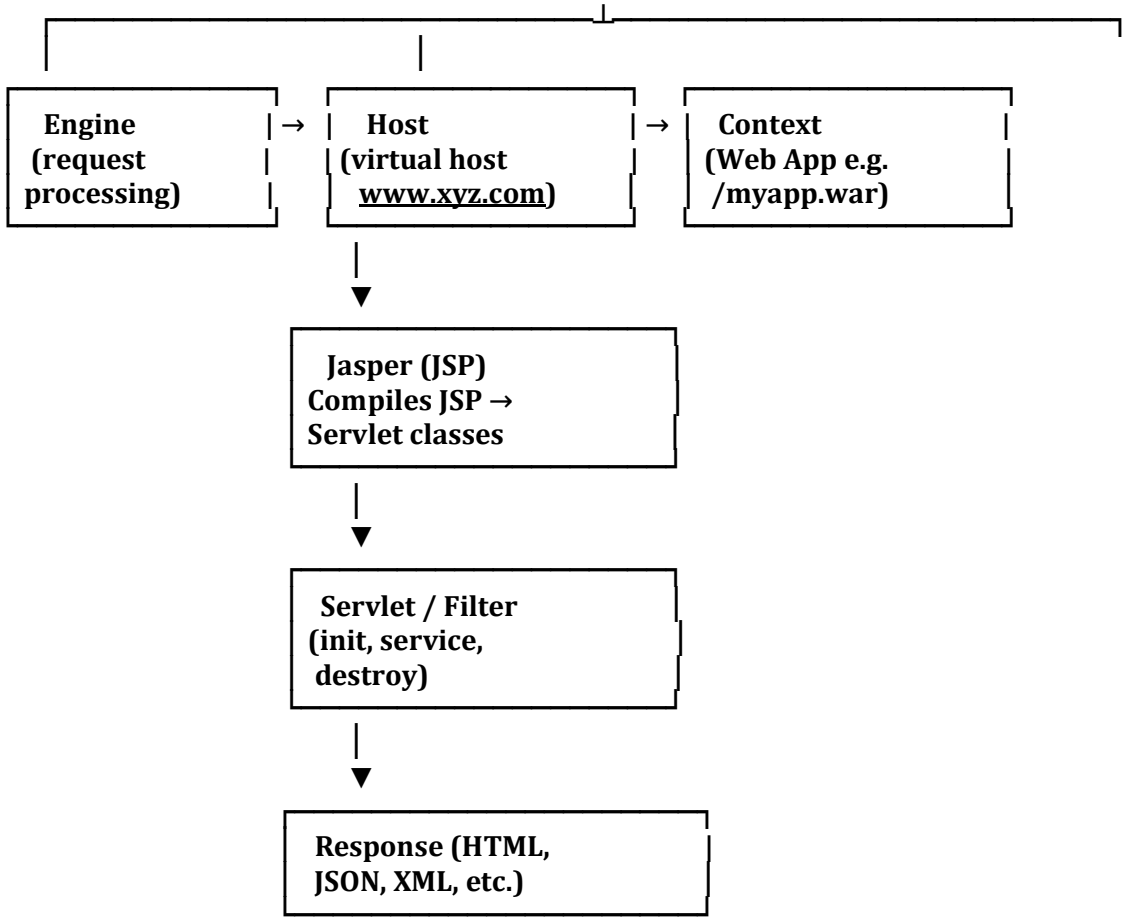
Manager App	A web-based admin tool for managing applications (deploy, undeploy, reload).

Tomcat Request Flow

1. **Client (Browser)** sends HTTP request.
2. **Coyote Connector** (listens on port 8080/8443) receives request.
3. Request goes to **Catalina Engine**, then → **Host** (virtual host), then → **Context** (web app).
4. **Jasper** compiles JSP into servlet if needed.
5. **Servlet Container** (Catalina) executes servlet service() → returns response.
6. **Coyote Connector** sends response back to client.

Tomcat Architecture Visual Diagram





Summary:

- **Coyote** = Front door (handles protocol & sockets).
- **Catalina** = Heart of Tomcat (Servlet container).
- **Jasper** = JSP compiler.
- **Host/Context** = Organize multiple apps.

Would you like me to also prepare a **side-by-side diagram of Tomcat Architecture vs Tomcat Life Cycle**, so you can show both together in interviews?

Perfect ⍰ This will give you an **interview-ready comparison —** Tomcat **Architecture Components** vs **Tomcat Life Cycle Stages** in a **side-by-side visual diagram**.

Tomcat Architecture vs Life Cycle

1. Architecture Components (What Tomcat Is Made Of)

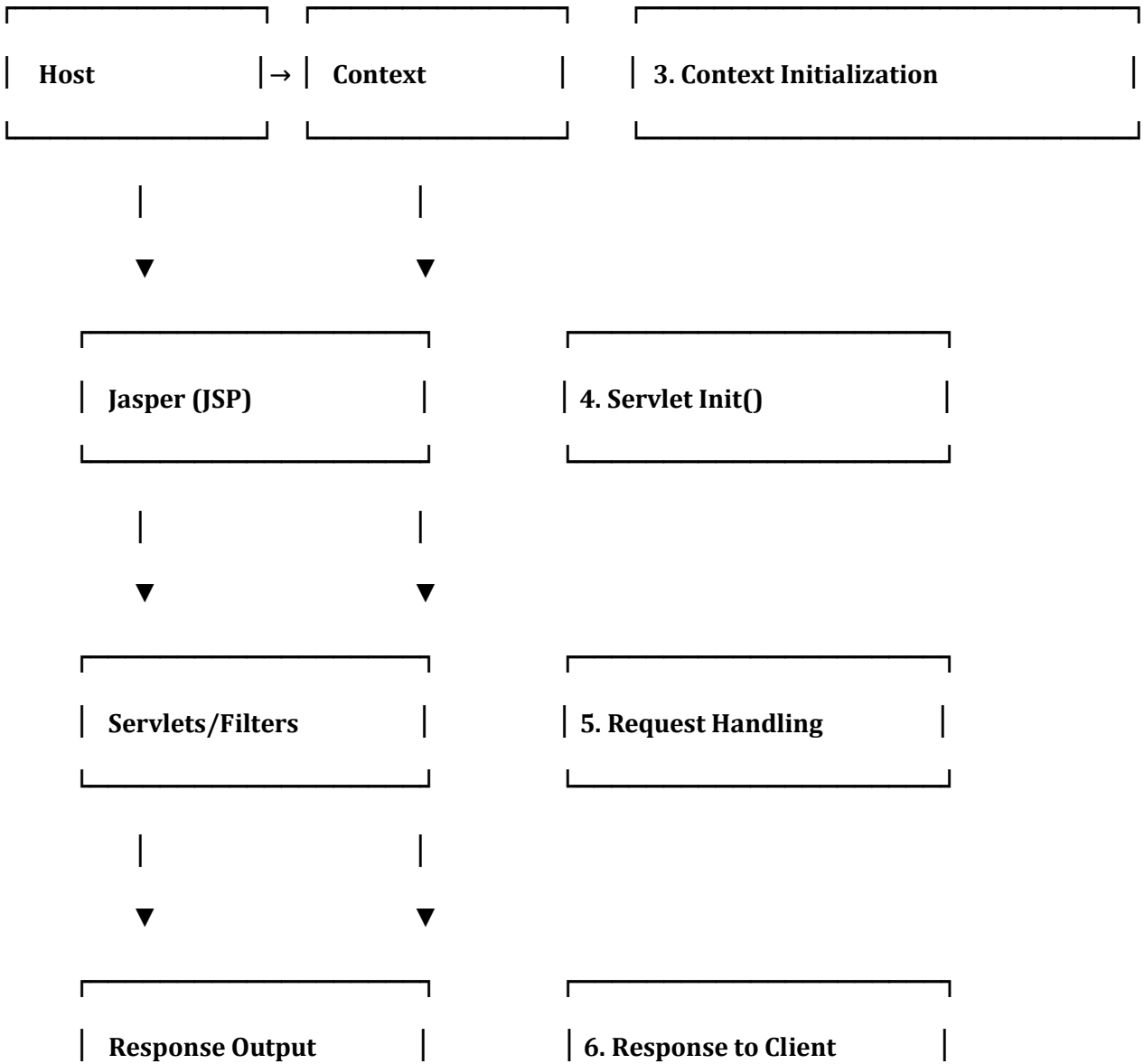
- **Coyote** → HTTP/AJP connector (receives client requests)
- **Catalina** → Core Servlet container (process requests)
- **Jasper** → JSP engine (compiles JSP → Servlets)
- **Host/Context** → Virtual hosts & web applications
- **Valves/Realms** → Security/authentication
- **Cluster** → Load balancing, session replication

2. Life Cycle (How Tomcat Works Step-by-Step)

- **Initialization** (Bootstrap, Catalina start)
- **Connector Startup** (Coyote listens on ports)
- **Context Initialization** (Deploy WAR, create webapp context)
- **Servlet Initialization** (init())
- **Request Processing** (service() → doGet/doPost)
- **Response Generation** (back to client via Coyote)
- **Shutdown** (destroy() and Catalina stop)

Side-by-Side Visual Diagram

```
┌──────────────────────────┐        ┌──────────────────────────┐
│ Tomcat Architecture      │        │ Tomcat Life Cycle        │
└──────────────────────────┘        └──────────────────────────┘
        │                 │
        ▼                 ▼
┌──────────────────────────┐        ┌──────────────────────────┐
│ Coyote (Connector)       │        │ 1. Initialization (Bootstrap) │
└──────────────────────────┘        └──────────────────────────┘
        │                 │
        ▼                 ▼
┌──────────────────────────┐        ┌──────────────────────────┐
│ Catalina (Servlet Engine) │        │ 2. Connector Startup (Coyote) │
└──────────────────────────┘        └──────────────────────────┘
        │                 │
        ▼                 ▼
```


Host
Context
3. Context Initialization
Jasper (JSP)
4. Servlet Init()
Servlets/Filters
5. Request Handling
Response Output
6. Response to Client

```
└──────────────────────┘          └──────────────────────┘

    │                  │
    ▼                  ▼

┌──────────────────────┐          ┌──────────────────────┐
│ Valves/Realms        │          │ 7. Shutdown/Destroy() │
└──────────────────────┘          └──────────────────────┘
```

Quick Interview Tip:

- If they ask **"What is Tomcat architecture?"** → Talk about **Coyote, Catalina, Jasper, Host/Context**.
- If they ask **"Explain Tomcat life cycle"** → Walk through **init → request → response → shutdown**.

Do you want me to also prepare this in a **one-page PDF cheat sheet with the diagrams** so you can keep it for interview prep?